

行情服务系统项目计划

1. 项目背景

建设一套高并发行情服务系统，对外提供股票行情快照、五档盘口、分笔成交、历史分笔、K线、复权数据、基础资料等查询能力。系统目标不是简单转发上游行情源请求，而是建设一套具备缓存、聚合、限流、连接池、多行情源节点调度、容错降级和可观测能力的行情服务平台。

本项目将基于前期行情源协议分析成果，建设统一的协议适配层和行情服务层，对外屏蔽底层行情源协议差异，对内通过多连接、多节点并发接入行情服务器，实现稳定、可扩展、可运维的行情数据服务。

2. 建设目标

2.1 性能目标

系统目标承载能力：

指标	目标
对外请求吞吐	1000 QPS
热点快照接口缓存命中响应	$p95 \leq 50ms$
普通快照接口响应	$p95 \leq 200ms$
K线/历史类接口响应	$p95 \leq 500ms$ ，优先缓存命中
服务可用性	$\geq 99.9\%$
热点行情缓存命中率	$\geq 80\%$ ，压测阶段根据业务分布校准
上游行情源请求削峰	对外 1000 QPS 不等于上游 1000 QPS，需通过缓存和批量聚合显著降低上游压力

2.2 功能目标

系统一期覆盖：

- 实时行情快照查询；
- 五档盘口查询；

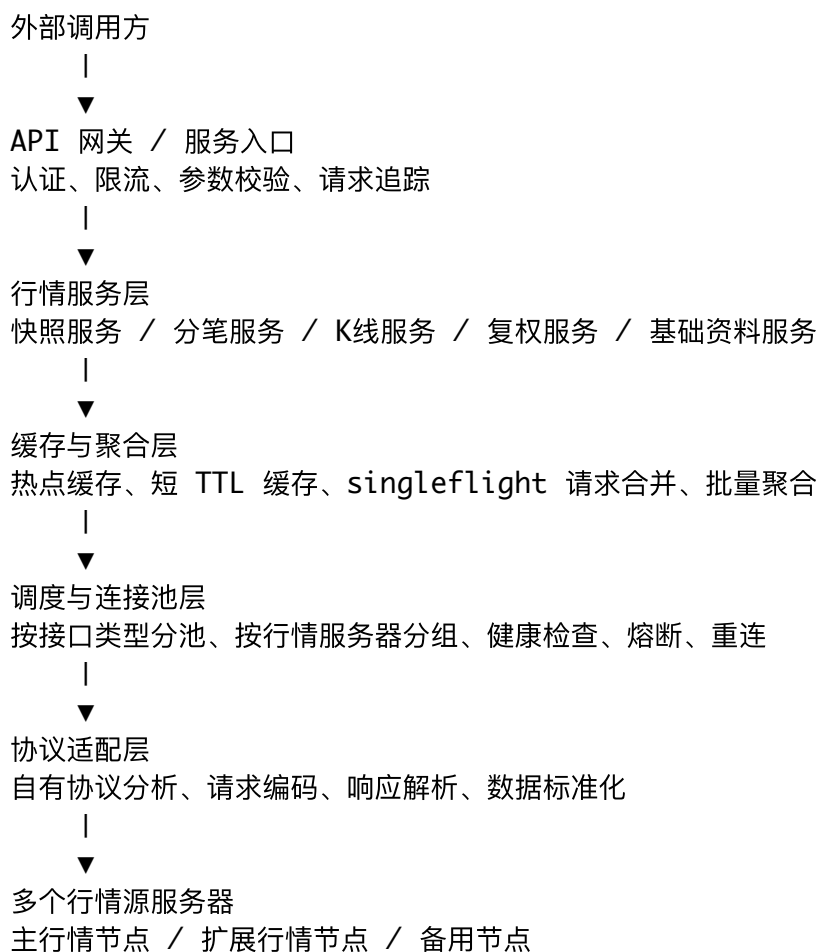
3. 批量行情查询；
4. 当日分笔成交查询；
5. 历史分笔成交查询；
6. 日 K / 分钟 K 查询；
7. 前复权 / 后复权 K 线查询；
8. 除权除息事件查询；
9. 股票基础信息查询；
10. 财务 / F10 类低频信息查询；
11. 行情源节点健康检查；
12. 多行情源服务器连接池；
13. 本地缓存、限流、熔断、降级；
14. 监控指标、日志追踪和压测报告。

2.3 架构目标

1. 对外 API 稳定，不暴露底层行情源协议细节；
2. 内部通过统一协议适配层封装行情源交互；
3. 支持同时连接多个行情服务器；
4. 支持按接口类型拆分连接池和并发控制；
5. 支持热点行情缓存和请求合并；
6. 支持后续替换或扩展其他行情源；
7. 支持灰度压测、限流保护和故障隔离。

3. 总体设计方案

3.1 总体架构



3.2 核心设计原则

1. 对外承载与上游请求解耦

系统承载 1000 QPS，并不代表每秒向行情源发起 1000 次请求。通过缓存、批量聚合、请求合并和后台刷新降低上游压力。

2. 按接口类型隔离资源

快照、分笔、历史分笔、K线、历史成交、F10、除权除息等接口耗时和响应体不同，必须使用不同连接池和并发上限，避免重接口拖垮实时快照服务。

3. 多行情服务器并行接入

底层同时维护多个行情服务器连接，支持按健康状态、延迟、错误率和负载进行调度。

4. 缓存优先，实时刷新补充

高频快照接口以短 TTL 热缓存为主；分笔、K线、复权、F10 等数据以本地缓存或持久化缓存为主。

5. 服务层自控，不依赖单连接能力

单个行情源连接不作为承载能力边界，系统通过连接池、节点池和服务层调度实现扩展。

6. 可观测、可压测、可降级
- 所有核心路径必须有指标、日志、链路追踪、错误分类和压测报告。

4. 模块设计

4.1 API 服务模块

职责：

- 提供 HTTP 或 gRPC API；
- 参数校验；
- 请求限流；
- 返回统一错误码；
- 生成请求追踪 ID；
- 屏蔽内部行情源协议和节点信息。

建议接口：

接口	说明
GET /api/v1/quotes	批量快照行情
GET /api/v1/orderbook	五档盘口
GET /api/v1/ticks	当日分笔成交
GET /api/v1/history-ticks	历史分笔成交
GET /api/v1/kline	K线
GET /api/v1/adjusted-kline	前/后复权 K线
GET /api/v1/xdxr	除权除息事件
GET /api/v1/securities	股票基础信息
GET /api/v1/finance	财务摘要
GET /api/v1/health	服务健康检查
GET /api/v1/metrics	监控指标，按部署方式决定是否暴露

4.2 行情服务模块

职责：

- 对接 API 层请求；
- 识别接口类型；
- 决定走缓存、数据库还是实时行情源；
- 统一返回标准行情模型。

核心服务：

服务	职责
QuoteService	实时快照、五档盘口、批量报价
TickService	当日分笔、历史分笔、逐笔成交分页与全量拉取
KlineService	日 K、分钟 K、历史 K 线、本地增量补齐
AdjustService	前复权、后复权、除权除息处理
SecurityService	股票代码、名称、市场、状态等基础信息
FinanceService	财务、F10、公司资料等低频数据
SourceHealthService	行情源节点健康、延迟、错误率管理

4.3 缓存与请求合并模块

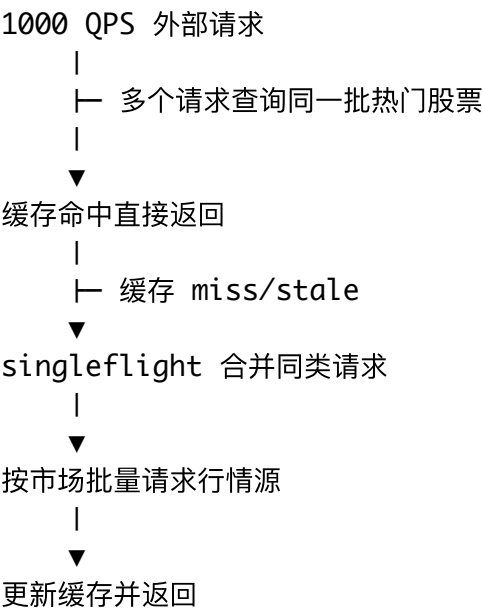
职责：

- 热点行情短 TTL 缓存；
- 请求合并；
- 批量聚合；
- 防止缓存击穿；
- 支持 stale-while-revalidate 策略。

建议策略：

数据类型	缓存策略
五档快照	100ms~1000ms TTL，热点标的后台刷新
批量报价	按市场和 symbol 分组缓存
当日分笔	短 TTL 或按页缓存，限制单 symbol 刷新频率
历史分笔	本地持久化缓存 + 按交易日增量补齐
日 K / 分钟 K	本地持久化缓存 + 增量补齐
复权 K 线	本地缓存，按复权类型和周期区分 key
除权除息	日级缓存，定时刷新
F10 / 财务	长 TTL 缓存，低频刷新

请求合并示例：



4.4 多行情源连接池模块

职责：

- 同时连接多个行情服务器；
- 按服务器维护连接组；
- 按接口类型维护连接池；
- 支持健康检查、重连、熔断、降级；
- 支持按延迟和错误率动态调度。

建议分池：

连接池	用途	特点
quote-pool	快照、五档盘口、批量报价	高频、低延迟、批量化
tick-pool	当日分笔成交	中高频、分页拉取、限制单标的刷新频率
history-pool	历史分笔、历史分时、历史成交	低并发、队列化、强缓存
kline-pool	K线	中低频、缓存优先
static-pool	F10、财务、文件类数据	很低并发、强缓存
adjust-pool	除权除息、复权基础数据	定时刷新，不进高频路径

多节点调度策略：

1. 启动时加载行情服务器列表；
2. 并发探测每个节点延迟和可用性；
3. 为可用节点建立连接池；

4. 按节点健康分数分配请求；
5. 节点失败后自动熔断；
6. 熔断节点定期半开探测；
7. 节点恢复后逐步恢复流量。

节点健康分数可参考：

```
health_score = latency_score * 0.4
               + success_rate_score * 0.4
               + recent_error_score * 0.2
```

4.5 协议适配模块

职责：

- 基于自有协议分析成果，实现行情源协议请求编码；
- 实现响应包解析；
- 处理压缩、价格编码、成交量编码、时间字段解析；
- 将不同协议返回统一转换为内部标准模型；
- 隔离底层行情源协议变化。

模块边界：

业务服务层只依赖 `Provider` 接口
`Provider` 内部封装协议适配实现
协议适配层不向外暴露原始协议结构

建议抽象：

```
type MarketDataProvider interface {
    GetQuotes(ctx context.Context, symbols []Symbol) ([]Quote,
    GetOrderBook(ctx context.Context, symbols []Symbol) ([]OrderBook,
    GetTicks(ctx context.Context, symbol Symbol, req TickRequest) ([]Tick,
    GetHistoryTicks(ctx context.Context, symbol Symbol, req HistoryTicksRequest) ([]Tick,
    GetKLine(ctx context.Context, symbol Symbol, req KLineRequest) ([]KLine,
    GetAdjustedKLine(ctx context.Context, symbol Symbol, req AdjustedKLineRequest) ([]KLine,
    GetXDXR(ctx context.Context, symbol Symbol) ([]XDXREvent,
    GetSecurityInfo(ctx context.Context, req SecurityQuery) (SecurityInfo,
}
```

4.6 本地数据存储模块

建议使用本地数据库或缓存系统承接低频和历史数据：

数据	存储策略
----	------

股票代码表	持久化，定时更新
交易日历	持久化，定时更新
K线	持久化，增量更新
当日分笔	短周期缓存，必要时落库
历史分笔	按交易日持久化或冷存储，按需回补
复权基础数据	持久化，日级更新
F10 / 财务	持久化，低频更新
热点快照	内存缓存 / Redis 短 TTL

一期可采用：

- 内存缓存承接热点快照；
- Redis 承接多实例共享缓存；
- MySQL / PostgreSQL 承接 K线、历史分笔、基础资料、复权基础数据；
- 后续如数据量增长，可引入 ClickHouse 或时序数据库。

4.7 可观测性模块

必须采集：

指标	说明
API QPS	对外请求量
API latency	p50 / p95 / p99
API error rate	错误率
cache hit rate	缓存命中率
upstream QPS	行情源请求量
upstream latency	行情源响应耗时
upstream error rate	行情源错误率
per-source health	每个行情服务器健康状态
reconnect count	重连次数
circuit breaker state	熔断状态
stale response ratio	返回旧缓存比例

错误分类：

- 连接失败；

- 读写超时；
- 响应解析失败；
- 返回数据不完整；
- 行情源节点不可用；
- 本地缓存异常；
- 数据库异常；
- 限流拒绝。

5. 1000 QPS 承载策略

5.1 对外 1000 QPS

通过以下手段承载：

1. 服务实例水平扩展；
2. API 层限流和隔离；
3. 热点数据短 TTL 缓存；
4. 请求合并；
5. 批量接口优先；
6. 本地历史数据缓存；
7. 多行情源节点连接池；
8. 服务熔断和降级。

5.2 上游请求削峰

目标不是向行情源打满 1000 QPS，而是将上游请求控制在安全范围内。

示例：

外部 1000 QPS
热点快照缓存命中率 80%
剩余 200 QPS 进入服务层处理
通过请求合并和批量查询，压缩为 20~80 QPS 上游请求

实际上游请求量需通过压测和行情源稳定性测试确定。

5.3 多行情服务器并发连接

假设可用行情服务器数量为 N，每个服务器维护 M 条连接：

总连接数 = $N \times M \times$ 接口池数量

示例初始配置：

类型	节点数	每节点连接数	总连接数
快照行情	5	4	20
当日分笔	3	2	6
K线	3	2	6
历史分笔/历史数据	2	1	2
F10/财务	2	1	2

上线后根据指标动态调整：

- 如果缓存命中高，可降低上游连接数；
- 如果单节点错误率升高，自动降低权重；
- 如果整体延迟升高，扩容服务实例或增加节点连接；
- 如果行情源出现限流，立即降低上游并发并启用 stale 缓存。

6. 高可用与降级设计

6.1 节点级容错

- 单个行情服务器失败，不影响整体服务；
- 失败节点自动熔断；
- 自动切换到其他可用节点；
- 后台定期探测恢复。

6.2 接口级隔离

- 快照接口优先级最高；
- K线接口中等优先级；
- 分笔、历史分笔、F10、财务、历史数据低优先级；
- 重接口不能占用快照连接池，历史分笔不能占用当日分笔连接池。

6.3 缓存降级

当行情源不可用时：

场景	降级策略
快照短暂失败	返回最近一次缓存，并标记数据时间
K线失败	返回本地已有数据，提示缺口未更新
分笔失败	返回最近缓存页或明确提示该标的分笔数据暂不可用
F10/财务失败	返回本地缓存数据

全部行情源不可用 返回明确错误码，并保留只读缓存服务

6.4 限流策略

建议限流层级：

- 1. 全局 QPS 限流；
- 2. 客户级限流；
- 3. 接口级限流；
- 4. symbol 级刷新频率限制；
- 5. 上游行情源节点级限流。

7. 数据模型设计

7.1 标准证券标识

Symbol = market + code

字段：

字段	说明
market	市场，例如 SH / SZ / BJ 等
code	证券代码
name	证券名称
type	股票、ETF、指数、债券等
status	正常、停牌、退市等

7.2 快照行情模型

字段	说明
symbol	标准证券标识
last_price	最新价
open	开盘价
high	最高价
low	最低价
pre_close	昨收价

volume	成交量
amount	成交额
bid_levels	买一至买五
ask_levels	卖一至卖五
quote_time	行情时间
source_time	源数据时间
cached	是否来自缓存

7.3 K线模型

字段	说明
symbol	标准证券标识
period	周期
adjust_type	不复权 / 前复权 / 后复权
time	K线时间
open	开盘价
high	最高价
low	最低价
close	收盘价
volume	成交量
amount	成交额
source	数据来源

7.4 分笔成交模型

字段	说明
symbol	标准证券标识
trade_time	成交时间
price	成交价
volume	成交量
amount	成交额
direction	买卖方向，按行情源能力标准化

sequence	分笔序号或页内顺序
source_date	交易日期，历史分笔必填
cached	是否来自缓存

7.5 除权除息模型

字段	说明
symbol	标准证券标识
event_date	事件日期
event_type	分红、送转、配股、扩缩股等
cash_dividend	现金分红
bonus_share	送转股比例
allotment_price	配股价
allotment_ratio	配股比例
raw_fields	原始扩展字段，便于追踪

8. 开发计划

8.1 项目周期

一期周期：**4 周**。
如需增加完整前端管理台、复杂权限、多租户计费或大规模历史数据回补，周期需延长。

8.2 阶段安排

阶段	周期	工作内容	交付物
阶段 1：需求确认与技术设计	第 1 周	明确接口范围、QPS 指标、部署环境、数据范围、行情源节点清单、缓存策略	需求规格说明、技术架构设计、接口草案
阶段 2：协议适配与连接池基础	第 1 周	完成行情源协议适配、请求编码、响应解析、多服务器连接、健康检查、基础连接池	协议适配模块、连接池模块、节点探测能力
阶段 3：行情核心接口开发	第 2	实现快照、五档盘口、批量报价、当日分笔、历史分笔、K线、复权 K	行情 API、统一数据模型、初版服务

周 线、基础信息接口			
阶段 4：缓存、聚合与限流	第 3 周	实现热点缓存、分笔分页缓存、请求合并、批量聚合、接口级限流、上游削峰策略	缓存模块、限流模块、批量聚合模块
阶段 5：低频数据与本地存储	第 4 周	实现除权除息、财务/F10、本地 K 线缓存、历史分笔缓存、基础资料持久化	数据库存储、低频数据接口、定时更新任务
阶段 6：可观测性与容错	第 4 周	实现监控指标、日志追踪、熔断、降级、节点健康看板数据	指标体系、熔断降级能力、运维文档
阶段 7：压测、调优与验收	第 4 周	完成 1000 QPS 压测、稳定性测试、故障演练、性能调优和验收文档	压测报告、验收报告、部署文档、交付版本